MSc Computer Science — Distributed Computing

Lab 06

Caching and Replication

Take-Home Assignment | Go Language | Docker Desktop

Item	Detail
What you build	A distributed LRU cache with write-through/write-back strategies and replication
Where you run it	Your own laptop — Docker Desktop required
Estimated time	4–5 hours
Prior lab code	Not required — fully independent
Submission	complete Moodle quiz
Questions	Post to Moodle forum

Background — Why Caching Matters

A cache sits between your application and a slow data source (database, API, CDN origin). It stores recently-accessed data so future requests can be served faster — without going to the slow source every time.

Without Cache	With Cache
Every request hits the database	Most requests served from cache (fast)
Response time: 50–200ms	Response time: <1ms for cache hit
Database overloaded at high traffic	Database load dramatically reduced
One failure point: the database	Cache absorbs traffic spikes

What You Are Building

A simplified version of systems like Redis Cluster or a CDN edge cache. Your system: origin (slow DB) → cache-primary → cache-replica1, cache-replica2 Clients read from cache first. On cache miss, origin is queried and result stored in cache. Writes go to cache using either write-through (safe) or write-back (fast) strategy. All writes replicate to replicas automatically.

Three Core Concepts

Concept	What it means	You implement
LRU Eviction	When cache is full, remove the Least Recently Used entry	evictLRU() — Tasks 1-4
Write Strategies	Write-through: update origin immediately. Write-back: update origin later.	WriteThrough/WriteBack — Tasks 5-7
Replication	Primary pushes writes to replicas so reads can be served from any node	Replicate/ApplyReplica — Tasks 8-10

Setup — Get Your Environment Running

> ⚠ **Do This First**
> Complete all setup steps before touching any code.

Step 1 — Install Docker Desktop

Your OS	Download link	Notes
Windows 10/11	https://docs.docker.com/desktop/install/windows-install/	Requires WSL2
macOS (Intel)	https://docs.docker.com/desktop/install/mac-install/	Intel chip version
macOS (Apple Silicon)	https://docs.docker.com/desktop/install/mac-install/	M1/M2/M3 version
Linux	https://docs.docker.com/desktop/install/linux-install/	Or Docker Engine + Compose plugin

```
docker --version
docker-compose --version
```

Step 2 — Download Lab 06 Files

```
unzip lab06-complete.zip
cd lab06-complete
ls
# Should show: student/  solution/  docker/  docs/  README.md
```

Step 3 — Build the Docker Image

```
docker build -t lab06 -f docker/Dockerfile .
# Takes 2-3 minutes on first run
# Expected last line: Successfully tagged lab06:latest
```

Step 4 — Start All Containers

```
docker-compose -f docker/docker-compose.yml up -d

docker ps | grep lab06
# Should show 5 containers:
#   lab06-origin          (simulated slow database)
#   lab06-cache-primary   (main cache, handles writes)
#   lab06-cache-replica1  (read replica)
#   lab06-cache-replica2  (read replica)
#   lab06-client          (run your tests from here)
```

Step 5 — Verify All Running

```
docker logs lab06-origin
# Expected: [ORIGIN] Database listening on port 9300 (read delay: 50ms)

docker logs lab06-cache-primary
# Expected: [CACHE] Initialised (capacity=10)
#           [RPC] Cache server listening on port 9200
```

> ⚠ **Before Testing**
> After the containers start, register the replicas with the primary. This tells the primary where to send writes for replication. Run this once after every docker-compose up:

```
# Register replicas with primary
docker exec lab06-client /lab06/cache/cache_bin -mode register \
    -primary cache-primary:9200 -replica cache-replica1:9200
docker exec lab06-client /lab06/cache/cache_bin -mode register \
    -primary cache-primary:9200 -replica cache-replica2:9200

# Expected:
# Registered replica cache-replica1:9200 with primary cache-primary:9200
# Registered replica cache-replica2:9200 with primary cache-primary:9200
```

Container Reference

Container	Role	Enter with
lab06-origin	Simulated slow database (50ms delay)	docker exec -it lab06-origin bash
lab06-cache-primary	Primary cache — edit code here	docker exec -it lab06-cache-primary bash
lab06-cache-replica1	Read replica 1	docker exec -it lab06-cache-replica1 bash
lab06-cache-replica2	Read replica 2	docker exec -it lab06-cache-replica2 bash
lab06-client	Run all client commands here	docker exec -it lab06-client bash

How to Edit, Recompile and Restart

```
docker exec -it lab06-cache-primary bash
cd /lab06/cache
nano cache.go          # edit
go build -o cache_bin .
pkill cache_bin
./cache_bin -mode cache -port 9200 -origin origin:9300 -capacity 10 &

# Must also rebuild replicas after editing shared code
docker exec lab06-cache-replica1 bash -c \
  'cd /lab06/cache && go build -o cache_bin . && pkill cache_bin; \
   ./cache_bin -mode cache -port 9200 -origin origin:9300 -capacity 10 &'
```

CLI Reference

```
# All commands run from inside lab06-client
docker exec lab06-client /lab06/cache/cache_bin [flags]

# Get a key (cache hit or origin fallback)
-mode get   -cache cache-primary:9200 -key <key>

# Set a key — write-through (safe, slower)
-mode set   -cache cache-primary:9200 -key <key> -value <value> -strategy write-
through

# Set a key — write-back (fast, risk of loss)
-mode set   -cache cache-primary:9200 -key <key> -value <value> -strategy write-
back

# Show cache statistics
-mode stats -cache cache-primary:9200

# Run benchmark (warmup test)
-mode bench -cache cache-primary:9200
```

Part 1 — LRU Cache (Tasks 1–4)

```
docker exec -it lab06-cache-primary bash
cd /lab06/cache
nano cache.go   # Tasks 1-4
```

Cache Tasks

Task 1 cache.go	`NewCache()` **[Easy]**	Initialise items map, set capacity, create NewStats(), print message
Task 2 cache.go	`Get()` **[Easy]**	Look up key, update LastAccessed on hit, call stats.Hit() or stats.Miss()
Task 3 cache.go	`Set()` **[Easy]**	If new key and cache full → evictLRU(), then store Entry with time.Now()
Task 4 cache.go	`evictLRU()` **[Hard]**	Loop entries, find oldest LastAccessed, delete it, call stats.Eviction()

> **Why LRU?**
> LRU keeps the entries you have accessed recently and discards ones you haven't touched in a while. The assumption is that recently-used data is likely to be needed again soon (temporal locality). Used in CPU caches, Redis, Memcached, browser caches, CDNs.

Test Part 1

```
# After recompiling — put 5 keys
for k in a b c d e; do
  docker exec lab06-client /lab06/cache/cache_bin -mode set \
    -cache cache-primary:9200 -key $k -value val_$k -strategy write-through
done

# Get them back — should show HIT for all
for k in a b c d e; do
  docker exec lab06-client /lab06/cache/cache_bin -mode get \
    -cache cache-primary:9200 -key $k
done

# Check stats
docker exec lab06-client /lab06/cache/cache_bin -mode stats -cache cache-
primary:9200
# Expected: Hits: 5  Misses: 0  Hit Rate: 100.0%
```

Part 2 — Write Strategies (Tasks 5–7)

```
docker exec -it lab06-cache-primary bash
cd /lab06/cache
nano strategy.go    # Tasks 5-7
```

Strategy Tasks

Task 5 strategy.go — `WriteThrough()` [Medium]
Write to origin FIRST, then cache with dirty=false — return error if origin fails

Task 6 strategy.go — `WriteBack()` [Medium]
Write to cache ONLY with dirty=true — return immediately without touching origin

Task 7 strategy.go — `flushDirty()` [Medium]
For each dirty key: get value, write to origin, MarkClean — background flush

Write-Through vs Write-Back — Side by Side

	Write-Through	Write-Back
Origin updated	Immediately	After flush interval (3s)
Write latency	Slower (waits for origin ~50ms)	Fast (cache only, immediate)
Data safety	Safe — origin always current	Risk — dirty data lost if cache crashes
Use case	Bank balances, inventory	Analytics counters, session tokens
dirty flag	false	true (until flushed)

Test Part 2

```
# Time a write-through write (should take ~50ms — waits for origin)
docker exec lab06-client /lab06/cache/cache_bin -mode set \
    -cache cache-primary:9200 -key balance -value 1000 -strategy write-through
# Watch latency in output

# Time a write-back write (should be <1ms — cache only)
docker exec lab06-client /lab06/cache/cache_bin -mode set \
    -cache cache-primary:9200 -key session -value abc123 -strategy write-back
# Watch latency in output

# Check broker logs — write-back key should show [dirty] in cache contents
docker logs lab06-cache-primary | grep dirty

# Wait 5 seconds for flush to run, then check logs again
sleep 5
docker logs lab06-cache-primary | grep FLUSH
# Expected: [FLUSH] key="session" flushed to origin
```

Part 3 — Replication (Tasks 8–11)

```
docker exec -it lab06-cache-primary bash
cd /lab06/cache
nano replication.go  # Tasks 8-10
nano rpc.go          # Task 11
```

Replication Tasks

Task 8 replication.go — **`RegisterReplica()`** [Easy]
Append replica address to the replicas slice

Task 9 replication.go — **`Replicate()`** [Medium]
For each replica, launch goroutine calling ApplyReplica RPC — fire and forget

Task 10 replication.go — **`ApplyReplica()`** [Easy]
Call c.Set(key, value, false) on this replica's cache

Task 11 rpc.go — **`Get, Set, ApplyReplica handlers`** [Medium]
Get: cache-hit or origin fallback. Set: route by strategy then Replicate. ApplyReplica: call ApplyReplica()

Part 4 — Stats (Tasks 12–13)

```
docker exec -it lab06-cache-primary bash
cd /lab06/cache
nano stats.go   # Tasks 12-13
```

Task 12 stats.go — **`Hit(), Miss(), Eviction()`** [Easy]
Increment counters using atomic.AddInt64 — thread-safe without a mutex

Task 13 stats.go — **`HitRate() + PrintStats()`** [Easy]
hits/(hits+misses) and formatted output table

Test Replication

```
# Register replicas first (if not already done)
docker exec lab06-client /lab06/cache/cache_bin -mode register \
    -primary cache-primary:9200 -replica cache-replica1:9200
docker exec lab06-client /lab06/cache/cache_bin -mode register \
    -primary cache-primary:9200 -replica cache-replica2:9200

# Write to primary
docker exec lab06-client /lab06/cache/cache_bin -mode set \
    -cache cache-primary:9200 -key city -value London -strategy write-through
```

```
# Read from replicas — both should have the value
docker exec lab06-client /lab06/cache/cache_bin -mode get -cache cache-
replica1:9200 -key city
docker exec lab06-client /lab06/cache/cache_bin -mode get -cache cache-
replica2:9200 -key city
# Both expected: key="city" value="London"
```

Experiments

> ⚠ **All 13 Tasks Must Be Complete First**
> Both caches and origin must be running. Re-register replicas after any docker-compose restart.

Experiment A — Hit Rate Warmup

Steps:

1. Run the built-in benchmark: docker exec lab06-client /lab06/cache/cache_bin -mode bench -cache cache-primary:9200
2. Observe Round 1 (cold cache) and Round 2 (warm cache) latencies
3. Record hit rate after each round

Observe and record:

-> What was the latency on cache miss (Round 1)? On cache hit (Round 2)?
-> What was the hit rate after Round 2?
-> How much faster was a cache hit compared to a cache miss?

Experiment B — Write-Through vs Write-Back Latency

Steps:

4. Set 5 keys using write-through, record the latency shown for each
5. Set 5 different keys using write-back, record the latency for each
6. Calculate average latency for each strategy

Observe and record:

-> Average write-through latency?
-> Average write-back latency?
-> How much faster was write-back? Why exactly?

Experiment C — LRU Eviction

Steps:

7. Restart containers for a clean cache: docker-compose -f docker/docker-compose.yml restart
8. Set 10 keys (key0 through key9) — this fills the cache exactly
9. Read key0 to make it recently used
10. Set an 11th key (key10) — this should trigger LRU eviction
11. Check docker logs lab06-cache-primary for the eviction message
12. Try to get key1 (should be a cache miss — it was the LRU)

Observe and record:

-> Which key was evicted? Was it key0 (recently read) or key1 (not read)?
-> What was the log message when eviction happened?
-> Get key1 after eviction — was it a cache miss or cache hit?

Experiment D — Replica Reads

Steps:

13. Write 5 keys to the primary using write-through

14. Read all 5 keys from cache-replica1
15. Read all 5 keys from cache-replica2
16. Check stats on all 3 nodes

Observe and record:

-> Did both replicas have all 5 keys immediately after writing?
-> What was the hit rate on each replica?
-> Check latency — are replica reads as fast as primary reads?

Experiment E — Replica Failure

Steps:

17. Write 5 keys to primary (all 3 nodes should have them)
18. Stop one replica: docker stop lab06-cache-replica1
19. Try to get all 5 keys from the stopped replica — record what happens
20. Try to get all 5 keys from the running replica (replica2)
21. Write 3 new keys while replica1 is stopped
22. Restart replica1: docker start lab06-cache-replica1
23. Try to get the 3 new keys from replica1

Observe and record:

-> What happened when you tried to read from the stopped replica?
-> Did the running replica (replica2) serve reads correctly?
-> After restart, did replica1 have the 3 keys written while it was down?
-> What does this tell you about replication consistency?

Discussion Questions

24. In Experiment B, write-back was significantly faster than write-through. Describe exactly what data would be permanently lost if the cache-primary container crashed immediately after a write-back write (before the 3-second flush). How does write-through prevent this? When would you choose each strategy?

Answer:

25. Your LRU eviction removes the least recently used key. Describe a realistic scenario where LRU gives poor cache performance (hint: think about access patterns). What alternative eviction strategy would work better for that scenario?

Answer:

26. In Experiment E, replica1 missed writes that happened while it was stopped. When it restarted, it did not automatically catch up. Describe how you would add a catch-up mechanism — what information would the primary need to store, and what would the replica do on reconnection?

Answer:

27. Your cache is an AP system — it stays available but can return stale data during replication lag. Lab 03 showed that CP systems refuse requests to stay consistent. Describe a scenario where your cache returning stale data causes a real problem. How would you change the architecture to make it CP?

Answer:

Results Recording

Experiment A and B — Latency

Measurement	Value
A — Cache miss latency (Round 1)	
A — Cache hit latency (Round 2)	
A — Hit rate after Round 2	
B — Average write-through latency	
B — Average write-back latency	
B — Speed difference (x times faster)	

Experiment C — Eviction

Measurement	Value
Which key was evicted?	
Was it the LRU key? (Yes/No)	
Getting evicted key — cache hit or miss?	

Experiment D and E — Replication

Measurement	Value
D — Both replicas had all 5 keys? (Yes/No)	
D — Replica hit rate	
E — Reading from stopped replica result	
E — After restart: replica had missed writes? (Yes/No)	

Submission

Item	Where	Format
Discussion questions	Moodle — Lab 06 Quiz	Type in Moodle
Results tables	Moodle — Lab 06 Quiz	Fill in Moodle form

Submission Checklist

- [ ] Cache hits/misses work correctly — hit rate shows in stats
- [ ] LRU eviction triggered when 11th key added to capacity-10 cache
- [ ] Write-through latency is ~50ms (waits for origin)
- [ ] Write-back latency is <1ms (cache only)
- [ ] Dirty keys flushed to origin after 3 seconds
- [ ] Both replicas receive writes immediately after primary write
- [ ] Experiments A-E completed and recorded
- [ ] 4 discussion questions answered on Moodle

Troubleshooting

Problem	Fix
NewCache returned nil	Task 1 not implemented — return &Cache{...}
Hit rate always 0%	Task 12 not implemented — Hit/Miss not incrementing counters
Get always returns not found	Task 2 not returning value — check cache lookup logic
Write-through same speed as write-back	Task 5 not calling origin — WriteThrough must call originClient.Set()
Dirty keys never flushed	Task 7 not implemented — flushDirty must call MarkClean after origin write
Replicas never receive writes	Task 9 not calling RPC — Replicate must callRPC to each replica
LRU evicts wrong key	Task 4 not finding oldest — check .Before() comparison
'not implemented' error on Set	Task 11 Set handler not routing by strategy
Replica reads always miss	Task 10 not calling c.Set() — ApplyReplica must store in local cache
Container won't start	docker logs <container> to see error

Go Quick Reference

time.Time Comparisons

```
import "time"

entry.LastAccessed = time.Now()   // update to current time

// Check if a is older than b
if a.LastAccessed.Before(b.LastAccessed) {
    // a is older (less recently used)
}
```

Atomic Counters

```
import "sync/atomic"

var counter int64

// Increment safely from any goroutine
atomic.AddInt64(&counter, 1)

// Read safely
val := atomic.LoadInt64(&counter)
```

LRU Pattern

```
// Find the least recently used entry
oldestKey := ""
var oldestTime time.Time
for key, entry := range c.items {
    if oldestKey == "" || entry.LastAccessed.Before(oldestTime) {
        oldestKey = key
        oldestTime = entry.LastAccessed
    }
}
// Delete it
delete(c.items, oldestKey)
```